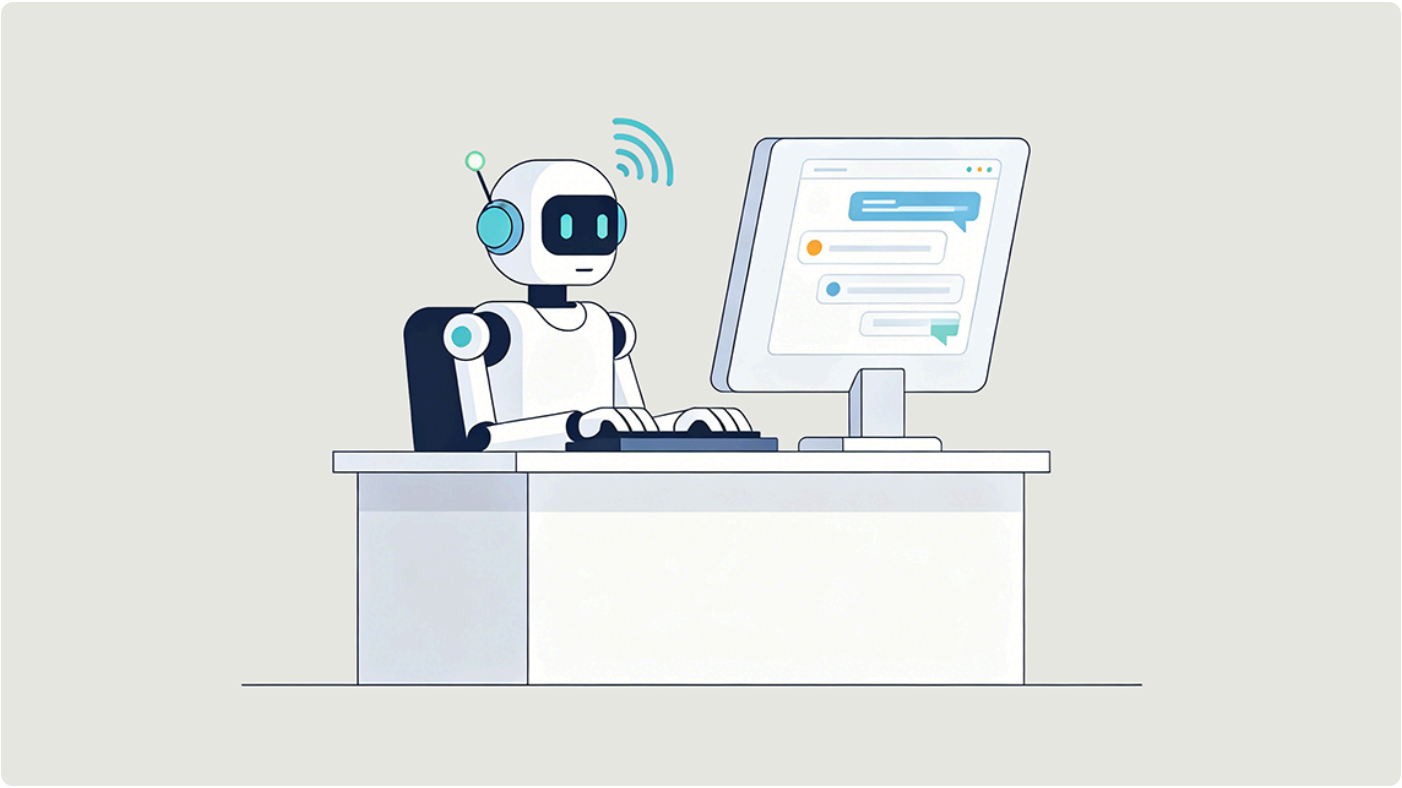
30 | 实操课：测试部署全流程演示

Robert · Claude Code 企业级全链路开发实战



你好，我是 Robert。

27 到 29 讲做了三件事：建质量体系、打包成可交付形态、加可观测性让系统从黑盒变透明。这一讲把它们放到真实操作里跑一遍。

建议先看完前三节课，再看这节实操课。图文讲明白为什么，视频展示做起来是什么感觉。

真实体验，我先说三句话

第一句：这三讲的方法论，我自己总结是“先想清楚，再动手”。但想清楚这件事本身，Claude Code 能帮你做大半。27 讲的核心链路分析，Claude Code 识别出来的风险点比我自己想的更全——有一个 `KnowledgeNodeExecutor` 里的 O(n) 线性搜索是它在做清单分析时主动发现的，我没要求它找。这种“你没问它也说”的时刻，是我觉得用对了的信号。

第二句：28 讲的部署，健康检查接口这个遗漏是 Claude Code 在咨询环节提醒我的。我没想到，它问了。如果我直接让它写 Dockerfile，它永远不会主动说“你这个接口还没有”。咨询模式和执行模式是两种节奏，分开用效果差很多。

第三句：29 讲的 Grafana Dashboard，Claude Code 在生成完后主动说 `DistributionSummary` 缺 `_bucket`，面板会空白。这个问题我不一定能发现。执行后自检这件事，Claude Code 不是每次都做，但在这几讲里它做了，我觉得值得单独说一下。

下面展开五个场景。



场景一：识别核心链路，写入 CLAUDE.md

所有测试的起点是知道测什么。先让 Claude Code 读完整个项目，输出核心链路清单和风险地图，你来确认，写进 CLAUDE.md。

提示词：

```
帮我分析这个系统，输出三个清单：

1. 核心链路清单（3-5条）
   每条链路：名称、涉及的模块和类、为什么是核心链路

2. 风险集中区域
   哪些模块/方法最容易出问题、出了问题影响最大
   每个风险点：风险类型（安全/并发/性能/数据一致性）、可能的失败场景

3. 测试重心建议
   基于前两条，测试精力应该往哪放
   哪些地方必须有测试覆盖，哪些可以先跳过

输出格式：结构化的 CLAUDE.md 片段，我直接复制进去。
```

Claude Code 读完代码后给出了 5 条核心链路和 7 个风险点，其中 2 个高危：`doStreamChat()` 里工作流路径抛 `RuntimeException` 时 user 消息落库但 assistant 消息缺失，以及 `selectRecentBySessionId` 的 SQL 取的是最旧消息而非最新。

拿到结果，对照自己的理解检查一遍。AI 的分析基于代码结构，你的判断来自业务理解，两者对比才能确认这份地图是准的。确认后写进 CLAUDE.md。这份地图会影响后面每一步：单测规范基于它生成，集成测试的 P0 优先级基于它划定。

接下来让 Claude Code 生成单测规范，同样基于这份地图，不是泛泛的通用规范。

提示词：

```
基于 CLAUDE.md 中的核心链路和风险地图，帮我生成 Hify 项目的单元测试规范。

规范需要覆盖：
1. 哪些代码必须写单测（结合核心链路判断）
2. 哪些代码不写单测、用集成测试替代（结合 Hify 的外部依赖特点）
3. 测试命名规范：should_[期望结果]_when_[输入条件]
4. 测试结构：Given-When-Then
5. mock 使用规范：什么时候 mock，什么时候不 mock
6. 断言规范：用 AssertJ，断言要有意义
7. 禁止事项：哪些写法不允许出现
```

```
输出格式：直接输出 CLAUDE.md 片段，我复制进去就能用。

```

规范里有一条禁止事项特别值得关注：禁止在测试里写业务逻辑，不要重复计算期望值，直接写字面量。这是 AI 写测试时最常见的坏味道，它会把被测的计算逻辑在测试里再实现一遍，测试通过只能说明两段逻辑一样错。规范明确写出来，后续 Claude Code 写任何测试代码都会自动遵守。

场景二：单测执行——看 Claude Code 如何纠正一份错误的计划

有了规范，先手动走一遍 `ProviderService` 的单测，跑通之后固化为 SKILL。

先规划，不直接写代码：

复制代码

```
深度分析 ProviderService 的 createProvider 方法。

告诉我：
1. 这个方法有哪些执行路径（正常路径 + 异常路径）
2. 每条路径的关键变量是什么
3. 哪些边界条件最容易出错

基于分析，给我一份测试计划：测哪些场景、每个场景验证什么断言。
先给计划，不要写代码。
```

Claude Code 给出了完整的执行路径树和 11 个测试场景，分 P0/P1/P2 三档。

然后我发了一份故意有三处错误的测试计划：

```
测试计划确认如下：
- 正常创建：provider name 唯一，验证返回 id 且数据库有记录
- 重复名称：抛出 BizException，错误码 PROVIDER_NAME_DUPLICATE
- apiKey 格式校验：不符合格式，抛出 BizException

按这个计划写单元测试。
约束：用 @MockBean mock 掉，只写单元测试。
```

Claude Code 没有照着错误的计划直接写代码，而是先指出了三个问题：

> “验证数据库有记录”在单测里做不到，`insert()` 被 mock 掉后数据不会真正写库，只能验证 `insert()` 被调用了一次。
>
> “apiKey 格式校验”在代码里根本不存在，这条测试没有被测对象。
>
> `@MockBean` 需要启动 Spring Context，和“只写单元测试”冲突，应该用 `@Mock + @InjectMocks + @ExtendWith(MockitoExtension.class)`。

高风险的任务先咨询再执行——测试计划、架构方案、接口设计，先让 Claude Code 过一遍，不要直接让它写代码。

确认后 Claude Code 生成两个文件，流程固化为 SKILL 放在 `.claude/skills/unit-test.md`，以后 `/单测AgentService.createAgent`，Claude Code 自动走完整流程。

场景三：集成测试——测试驱动修复

集成测试是 Hify 的主力：Spring Boot Test 完整启动，mock 掉外部 API，打真实 HTTP 请求，走完从 Controller 到数据库的完整链路。

先规划清单：

```
基于 CLAUDE.md 中的核心链路和风险地图，帮我规划 Hify 的集成测试清单。

测试范围：Spring Boot Test + MockMvc，mock 掉 LLM API 和 MCP Server，
```

```
测试 Hify 自身从 Controller 到数据库的完整链路。

按优先级分三档：
P0：核心链路，必须覆盖
P1：主要功能，应该覆盖
P2：边缘场景，有余力再做

每条给出：测什么、验证什么、为什么这个优先级。
不要写代码，先给清单。
```

Claude Code 先检查了 `application-mock.yml`，发现 H2 内存库和 `MockProviderAdapter` 已经就位，不需要额外搭建技术基础。

清单里有一条 IT-P0-03（对话上下文多轮正确性）值得单独演示，因为它对应一个已知 bug：

复制代码

```
写 IT-P0-03：对话上下文多轮正确性。

这条测试对应一个已知 bug，先写测试，测试应该是红的，然后再修 bug。

测试步骤：
1. 同一 session 依次发 3 条消息（"第一条"、"第二条"、"第三条"）
2. 在第 3 条发送时，拦截 MockProviderAdapter 收到的 ChatRequest

验证 MockProviderAdapter 第 3 次收到的 messages 数组：
① 包含"第一条"和"第二条"的历史消息
② 历史消息按时间升序排列（先旧后新，符合 LLM 期望）
③ 最后一条是"第三条"

这条测试现在应该失败，因为 selectRecentBySessionId 的 SQL 是
ORDER BY created_at ASC LIMIT，取的是最旧的消息而非最新的。
测试变红之后，再去修 SQL，修完测试变绿。
```

先写测试让它红，验证 SQL 取的确实是最旧消息；然后修 SQL（`ASC` 改 `DESC`）；测试从红变绿，bug 才算真正被修复。这是测试驱动修复，不是修完再补测试。

流程固化为 `.claude/skills/integration-test.md`，后续 `/集成测试Agent模块`，流程自动复用。

场景四：部署——咨询先行

本地部署、Docker 部署、K8s 部署，三种形态的顺序是从简单到复杂。但先做任何一个之前，咨询环节不能跳过。

```
Hify 是一个 Spring Boot + Vue 的 AI 应用，
后端调用外部 LLM API，依赖 MySQL、Redis、pgvector，
目标用户是企业内部团队，规模从几人到几十人不等。

我想把它做成可交付的形态，方便部署到不同环境。
帮我分析应该支持哪些部署形态，以及每种形态需要提前准备什么。
不要写任何代码，先给我分析。
```

Claude Code 给出三种形态对比和建议优先级，还主动问了四个确认问题。第四个是我没想到的：当前有没有健康检查接口？tar.gz、Docker、K8s 三种形态都需要它。没有就先补。

补接口的提示词：

```
帮 Hify 加一个健康检查接口 GET /api/v1/health，
返回 HTTP 200 和 {"code":0,"data":"ok"}。

这个接口会被三个地方用到：
- 本地部署：启动后 curl 确认服务起来了
- Docker：healthcheck 指令
- K8s：liveness 和 readiness 探针

接口本身只需要返回 ok，不需要检查数据库连接。
```

Claude Code 先扫描项目，发现 `HealthController.java` 已经存在，但格式不一致。它主动给出两个选项，而不是自作主张改全局格式。这是咨询模式和执行模式结合的正确节奏。

本地部署提示词：

```
帮 Hify 生成本地部署的打包脚本。

要求：
- 产物是一个 tar 包，包含：后端 jar、前端 dist 目录、start.sh、stop.sh、配置模板 applicat
- 目标机器已有 Java 环境，不需要打包 JDK
- start.sh 支持通过环境变量或配置文件注入 MySQL、Redis、pgvector 的连接信息
- stop.sh 优雅停止，等待进程退出
- Makefile 加 package 命令，一键打包

不需要包含 MySQL、Redis、pgvector，它们是外部服务。
```

Claude Code 先读了现有的 `start.sh`，发现是开发模式（跑 `mvn build + npm dev server`），告诉你需要完全重写，而不是在上面改。新的 `start.sh` 启动后轮询 `/api/v1/health`，就绪了打印成功，失败了打印最后 30 行日志告诉你哪里出了问题。

Docker 部署提示词分三步走：先后端 Dockerfile，再前端 Dockerfile，最后 `docker-compose.yml`。前端的提示词有两个关键约束不能省：

```
帮我写 Hify 前端的 Dockerfile。

情况说明：
- Vue 3 项目，npm run build 打包
- 用 Nginx 托管静态文件
- 前端需要把 /api 请求反向代理到后端

特别注意：
- Hify 有流式响应（SSE），Nginx 需要关闭缓冲
- LLM 调用可能很慢，超时时间要够长
```

`proxy_buffering off` 是 SSE 的关键。不说这个约束，Claude Code 生成的 Nginx 配置会把事件攒批后一次性发出，用户看不到打字机效果。

K8s 的核心提示词和验收不在这里展开，源码里都有，部署顺序看 29 讲的执行命令就够了。

场景五：可观测性——从黑盒到透明

可观测性这个领域知识面很广，先咨询不是因为你不懂，是因为有些架构决策改晚了代价极高。

```
Hify 是一个 Spring Boot + Vue 的 AI 应用，模块化单体架构，部署在 K8s 上。
核心功能是 LLM 对话，依赖 MySQL、Redis、pgvector，调用外部 LLM API 和 MCP Server。

我想加可观测性，帮我分析：
1. 应该做哪些，为什么
2. 哪些一期不需要做，理由是什么
3. 有没有什么需要提前考虑的架构决策

不要写代码，先给分析。
```

Claude Code 给出了三个架构决策：Actuator 要独立端口隔离（改晚了要动 K8s Service 和 Nginx）；traceId 要趁早进 MDC（改晚了要改所有日志打点代码）；日志字段规范要趁早定（改晚了所有告警规则都要跟着改）。这三个决策你不一定自己想的到，但它们决定了后续扩展的成本。

结构化日志的提示词：

```
帮 Hify 配置结构化日志，同时引入 OpenTelemetry 生成 traceId。

要求：
1. 日志格式是 JSON，包含 timestamp、level、traceId、thread、logger、message
2. 同一次请求的所有日志共享同一个 traceId，可以用 traceId 过滤出完整链路
3. traceId 通过 OpenTelemetry 生成，为后续链路追踪预留扩展点
4. 对话链路的关键节点要有日志：请求进入、LLM 调用开始/结束、工具调用、异常
5. 日志输出到 stdout，由 K8s 采集

不需要接 Jaeger 或 Zipkin，一期只用 traceId 串联日志。
```

Claude Code 读项目后发现三个问题：MDC 在新线程（`llmExecutor`）里会丢失；`logback-spring.xml` 是手拼 JSON 字符串，有转义风险；`ChatServiceImpl` 日志稀疏。这三个问题都是提示词里没说的，Claude Code 自己读代码发现的。新建了 `MdcTaskWrapper` 解决跨线程丢失，所有提交到 `llmExecutor` 的任务都用它包一层。

Grafana Dashboard 的提示词：

```
帮我生成 Hify 的 Grafana Dashboard JSON。

Hify 暴露了以下指标：
- hify_chat_requests_total (label: agent_id)
- hify_chat_duration_ms (DistributionSummary, label: agent_id)
- hify_llm_calls_total (label: provider, model, success)
- hify_llm_duration_ms (DistributionSummary, label: provider, model)
- hify_circuit_breaker_state (label: provider, 0=CLOSED 1=OPEN 2=HALF_OPEN)
- hify_mcp_tool_calls_total (label: tool, success)

Dashboard 需要包含以下面板：
1. 对话量（QPS 曲线，按 agent_id 分组）
2. 对话延迟（P50/P95/P99）
3. LLM 调用成功率（按 provider 分组）
4. LLM 调用延迟（P95，按 provider 分组）
5. 熔断器状态（各 Provider 当前状态）
6. MCP 工具调用成功率

输出完整的 Grafana Dashboard JSON，我直接导入使用。
```

Claude Code 在生成前主动发现指标名有误；生成后主动发现 `DistributionSummary` 缺 `_bucket`，询问是否补加 `.publishPercentileHistogram(true)`。两步缺一不可，没有第一步，Dashboard 指标对不上；没有第二步，P95/P99 面板全是空白，你都不知道为什么。

告警策略的提示词：

```
基于 Hify 当前暴露的指标，帮我梳理告警策略。

```

```
Hify 的业务背景：企业内部智能客服，工作时间使用，非 24 小时高可用场景。
用户规模：几十人并发。

帮我整理：
1. 哪些指标需要告警
2. 每条告警的阈值建议和触发条件
3. 告警级别（P0 立即响应 / P1 30 分钟内 / P2 次日处理）
4. 理由

不要写 Grafana 配置，先给策略清单。
```

Claude Code 给出的清单里有专门的“不建议告警”部分：熔断器 HALF_OPEN 状态是正常恢复流程，加了只会误判；非工作时间的任何 P1/P2，内部工具夜间无人使用不需要 on-call。告警太多会产生疲劳，不建议告警这部分和正面清单同样重要。

四条核心动作

这三讲反复出现了几个模式，值得单独记下来。

先咨询，再执行。部署形态讨论里发现健康检查遗漏，可观测性讨论里给出三个高代价架构决策，测试分析里识别风险地图。这些信息如果直接让 Claude Code 执行，它永远不会主动告诉你。

Claude Code 先读代码再动手。每个执行步骤里，Claude Code 在写代码之前都先扫了项目。发现 `start.sh` 是开发模式需要重写，发现 MDC 跨线程丢失，发现熔断器不走 Resilience4j Registry，发现健康检查接口已存在但格式不一致。这些问题如果直接执行，会产生能跑但有缺陷的代码。

执行后会自检。Grafana Dashboard 生成完后，Claude Code 主动发现 `DistributionSummary` 缺 `_bucket`。测试计划有问题，执行前先指出。端口写错了，按实际配置生成而不是按提示词盲目执行。

先写失败的测试，再修 bug。IT-P0-03 是这三讲里我觉得最重要的一条操作：测试变红才证明你找对了问题，变绿才证明你真正修好了。这个顺序不能反过来。

总结

测试部署篇结束了。Hify 从能跑的代码，变成了有质量保障、可交付、可观测的系统。

这三讲做完，你手上有四样东西：核心链路地图写进了 CLAUDE.md，单测 SKILL 和集成测试 SKILL 各就位一个，三种部署形态都能跑，Grafana 大盘和告警策略都配好了。

后面怎么用，你来定。如果你工作里有一个老项目，现在可以用场景一的提示词跑一遍，让 Claude Code 识别核心链路，对比你自己的判断，看两者有没有差异。差异的地方，往往就是你接下来该把精力放进去的地方。

期待你与我分享自己的体验。如果今天的课程让你有所收获，也欢迎转发给有需要的朋友，邀请他来一起学习，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。 页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

精选留言

> 由作者筛选后的优质留言将会公开显示，欢迎踊跃留言。